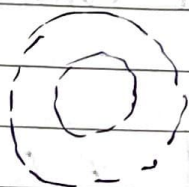


## Week-6 ADS

### 4 Spectral Clustering : Algorithm



$\Rightarrow$  nodes of the graph. Put wts. on edges. wts.  $\propto$  distances

$$G = (V, E, W)$$

$$L = D - W$$

Laplacian Matrix

$$D_j = \sum_i W_{ij}$$

$$\min_{f \in \mathbb{R}^n} \|f\|^T L f$$

$$\|f\|^2 = 1 \quad \& \quad f^T \mathbf{1} = 0$$

$\rightarrow f^+$   $\rightarrow$  apply KMeans.

or threshold

at certain value (eg. 0.5)

first solve it

& then see how to get for  $k$ -clusters.

$f^+$  is the eigen vector of  $L$  corresponding to second smallest eigenvalue ( $\lambda_2$ )

The smallest eigenvalue of  $L$  is 0 & the corresponding eigenvector is all 1's vector.

- In general, better to use  $k$ -dimensional embeddings.

$$L = D - W$$

eigen decomposition  $\rightarrow$

$$\begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} \begin{bmatrix} \lambda_1 = 0 \\ \vdots \\ \lambda_k \end{bmatrix} \begin{bmatrix} b_1 \\ \vdots \\ b_k \end{bmatrix}$$

$x_i \in \mathbb{R}^d \rightarrow [b_{21} \ b_{31} \ \dots \ b_{k1}]$  it tells us how to capture the diff. clusters.

- we get diff. views of our clusters  $\Rightarrow$  then apply  $k$ -means on the  $k$ -dimensional embeddings.  
We could also, normalize the embeddings such that their length is 1.

$$x_i \in \mathbb{R}^d \rightarrow \left[ \begin{array}{c} f_{1i} \quad f_{2i} \quad \dots \end{array} \right] \quad \downarrow \quad \hat{x}_i \in \mathbb{R}^k$$

This is the spectral clustering algo.  
(we are currently using unnormalized laplacian.)

$$v_i = \frac{x_i}{\|x_i\|} \in \mathbb{R}^k$$

## L2 Spectral Clustering at Scale: Specifics - 1

- Now, we have to deal with scale w.r.t Spectral Clustering.

for dense graphs  $\Rightarrow$  it is computationally expensive  
 $O(n^2)$  edges

sparse graphs  $\Rightarrow$  faster algs. are available  
 $\ll O(n^2)$  edges to solve eigenvectors.

one other soln<sup>n</sup> is KNN graphs

$\downarrow$   
all wts. do not matter,  
only  $k$ -nearest wts. are considered.



## - Graph-Sparsification

→ sparsify the dense graph, remove some edges: Can't be done randomly, as the results are dependent on it.

$$\min_{\|f\|=1} \left| f^T L f \right| \quad \text{this is very imp.}$$

We want to preserve this quantity after sparsification.

Let's say  $f$  is an  $e$ -vector of  $L$  with  $e$ -value  $\lambda$ .

$$(D-w) \quad f^T L f = f^T (\lambda f) = \lambda f^T f = \lambda \|f\|^2 = \lambda$$

## - Spectral approximation of a graph

$G = (V, E, W)$  be a graph.  
 $FCE \leftarrow H = (V, F, W')$  is a spectral sparsifier of  $G$   
 if  $\forall f \in \mathbb{R}^{|V|}$ ,

$$(1 - \epsilon) f^T L_G f \leq f^T L_H f \leq (1 + \epsilon) f^T L_G f$$

- We have to see how to get this sparse graph  $H$ .

Sparsification by down-sampling method.

→ sample each edge  $(u, v) \in E$  with probab.  $p_{uv}$

→ if the edge has to be retained, then

$$w'_{uv} = \frac{w_{uv}}{p_{uv}}$$

→ it is random

$$- E \left[ f^T L_H f \right] = E \left[ \sum_{i,j \in V} (f_i - f_j)^2 w'_{ij} \right]$$

$$D_H = W_H$$



$$= \sum_{(i,j) \in E} p_{ij} (f_i - f_j)^2 \frac{w_{ij}}{p_{ij}}$$

$$= \sum_{(i,j) \in E} (f_i - f_j)^2 w_{ij} = f^T L_G f$$

This process is also known as importance sampling.

$$E[f^T L_H f] = f^T L_G f$$

on an avg. it is accurate, but I want this to accurate on the first time itself.

### L3 Spectral Sparsification - 2

- How to choose the probabilities for edges?

↳ "Effective Resistance"

$$p_{uv} = \underbrace{(e_u - e_v)^T}_{\text{effective resistance of the edge (u,v)}} L_G^+ \underbrace{(e_u - e_v)}_{\text{pseudo-inverse of the Laplacian}}$$

effective resistance

of the edge (u,v)

$$e_i = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

pseudo-inverse of the Laplacian

- we can show using above,

→ with high probab.

$$(1-\epsilon) f^T L_G f \leq f^T L_H f \leq (1+\epsilon) f^T L_G f$$

\_/\_/\_

If  $p_{uv} \propto R_{\text{eff}}(u, v)$  - then

$$|F| = O\left(\frac{|V| \log |V|}{\epsilon^2}\right)$$

$\hookrightarrow O(n \log n)$  no. of data pts.

- Fix an edge  $(i, j)$

$$R_{\text{eff}}(i, j) = e_{ij}^T L^+ e_{ij} \quad \left[ \begin{array}{l} e_{ij} = e_i - e_j \\ L^+ = L_{\text{G}}^+ \end{array} \right]$$

$$= e_{ij}^T \underbrace{L^+ L}_{I} L^+ e_{ij}$$

$$= e_{ij}^T L^+ \underbrace{U \Sigma V^T}_L L^+ e_{ij}$$

$$= \left\| \underbrace{e_{ij} L^+}_{\text{some vector}} \underbrace{U \Sigma^{1/2}}_{\text{some matrix}} \right\|^2$$

$$= \left\| \Sigma^{1/2} U^T L^+ e_i - \Sigma^{1/2} U^T L^+ e_j \right\|^2$$

we are just calculating the distance b/w. the edges.

we can now use JL (+ sth more) to get approximate effective resistances quickly.

#### L4 Spectral Clustering at Scale: Summary

Input: Dataset  $\{x_1, \dots, x_n\}$ ,  $k$

$\rightarrow$  Compute  $G = (V, E, W)$



- compute  $R_{\text{eff}}$  approx  $\forall (i,j) \in E$
- sample using  $p_{ij} \propto \hat{R}_{\text{eff}}(i,j)$  to create  $H$
- compute  $L_H$
- use sparse e-vectors solvers to get  $f_1, \dots, f_k$

$$\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_k$$

- normalize rows of  $f$  to get the normalized embeddings.
- run k-means on the embeddings to get the final clusters.